
Terms of Reference (RFQ)

Digital Solutions Expo 2026 – Event-Based Recruitment Coordination &

Interview Management Platform

Date: Thursday, 18 June 2026

Time: 9:00 AM – 5:00 PM

Venue: National ICT Innovation Hub, Kampala, Uganda

1. Background

Refactory Academy, in partnership with Outbox (U) Ltd, UNCDF and the Mastercard Foundation, is hosting the **Digital Solutions Expo 2026** under the theme “**Tech Made in Uganda.**” This one-day exhibition showcases market-ready digital solutions built by Uganda's homegrown tech talent and facilitates direct engagement between developers and industry stakeholders.

2. Objectives of This RFQ

Refactory Digital Solutions is commissioning the design and development of a centralized, web-based Event Recruitment Coordination and Interview Management Platform. This platform will serve as the operational backbone for physical hiring events where employers interview developer candidates on-site during a live event day.

The platform will be used before, during, and after each recruitment event. Pre-event, it manages employer job submissions, candidate registration, and interview scheduling. During the event, it coordinates real-time interview flow across multiple employer booths. Post-event, it archives outcomes and provides operational reports.

Employers submit roles in advance on our platform, and Talent is pre-screened and matched. The Talent Lounge will function as a controlled, high-trust environment designed to convert talent visibility into real employment, project, and career outcomes.

Experience Design

Employers submit: Open roles, and Skill requirements.

Refactory conducts: Talent pre-screening, and Role-talent matching

Interview schedules are: Pre-curated and pre-booked, using a digital tool

- ◆ This platform is NOT a traditional ATS, job board, or recruitment ERP.
- ◆ It is an event coordination and interview-scheduling operations system.
- ◆ Core value: intelligent scheduling · conflict prevention · real-time visibility · administrative efficiency.

The system will replace a fully manual coordination workflow that is currently burdened by scheduling conflicts, double bookings, missed interviews, communication delays, and heavy administrative overhead. The goal is a single, fast, operationally reliable platform that eliminates these failure points.

2. Problem Statement

2.1 Current Manual Workflow

The existing process involves seven manually executed steps:

- Employers submit job descriptions (JDs) via email or form.
- Admins curate candidate CVs manually against each JD.
- Admins email curated CVs to employers.
- Employers select preferred candidates and notify admins.
- Admins manually construct interview schedules.
- Admins coordinate candidate movement between booths during the event.
- Reminders and real-time updates are communicated manually via phone or messaging apps.

2.2 Identified Failure Points

Failure Point	Impact
Scheduling conflicts	Multiple candidates booked in the same slot
Double bookings	Same candidate assigned to two simultaneous interviews
Missed interviews	No automated reminders; candidates lose track of time
Poor visibility	No live operational view for admins or employers

Communication delays	Manual coordination over WhatsApp/email is slow
Operational overhead	Admins spend the entire event day firefighting manually
No audit trail	No structured record of interview outcomes or no-shows

3. Product Vision and Objectives

Build a centralised operational platform that:

- Allows employers to submit hiring requirements and define interview capacity.
- Allows admins to curate, filter, and assign candidates to employer shortlists.
- Allows employers to review curated candidates and select interviewees.
- Automatically generates conflict-free interview schedules.
- Coordinates physical interview flow across booths during live events.
- Sends automated, multi-channel reminders to all parties.
- Provides live operational visibility to admins on event day.
- Records interview states and outcomes for post-event reporting.

The platform should resemble event coordination software and lightweight scheduling systems — NOT heavy enterprise HR systems or complex ATS software.

4. User Roles and Permissions

Role	Access Scope	Key Capabilities
Admin	Full access to all modules	Schedule management, candidate curation, live dashboard, notifications, reporting
Employer	Scoped to own event participation	Submit JDs, review curated candidates, select interviewees, view own schedule
Student / Developer	Own profile and schedule only	Register, upload CV, view interview schedule, receive notifications

Super Admin	Platform configuration	Manage events, users, system settings, and analytics
-------------	------------------------	--

5. Functional Requirements

5.1 Authentication & Role Management

- Role-based access control (Admin / Employer / Student / Super Admin).
- Secure authentication: email + password with optional magic link.
- JWT session tokens with configurable expiry.
- Password reset via email OTP.
- Minimal onboarding: employer and student registration must be completed in under 3 minutes.

5.2 Employer Hiring Request Module

Employers can:

- Submit job descriptions (JDs) with required skills, seniority level, and role title.
- Define maximum interview capacity (number of candidates per slot).
- Define available interview slots (start time, end time, duration).
- Specify preferred booth/table assignment.
- Review admin-curated candidate shortlists.
- Select or deselect candidates from the shortlist.
- View their confirmed interview schedule.

5.3 Candidate (Student/Developer) Module

Students can:

- Create a lightweight profile: name, email, phone, skills, experience level.
- Upload a CV (PDF, max 5MB).
- Add portfolio/GitHub links.
- View assigned interview schedule with booth locations and times.
- Confirm or flag conflicts in their schedule.

Admins can:

- Approve or reject candidate registrations.
- Filter candidates by skills, experience level, availability.

- Curate candidate pools per employer JD.
- Bulk-assign candidates to employer shortlists.

5.4 Candidate Matching & Curation Workflow

The curation workflow follows four sequential steps:

- Employer submits a hiring request with JD and slot availability.
- Admin reviews registered candidates, filters by required skills, and curates a shortlist per employer.
- Employer reviews the curated shortlist and selects confirmed interviewees.
- System assigns selected candidates to available slots and generates the interview schedule.

5.5 Interview Scheduling Engine

The scheduling engine must:

- Allocate time slots to (candidate, employer) pairs without overlap.
- Enforce hard constraints: no double-booking per candidate or per employer booth.
- Respect interview durations defined by each employer.
- Support buffer time between consecutive interviews at the same booth.
- Allow manual rescheduling with automatic conflict re-check.
- Support cancellation with slot release and optional reassignment.
- Track each interview through defined state transitions.

State	Definition	Trigger
Scheduled	Interview created, not yet confirmed	Initial state post-assignment
Confirmed	Candidate and employer acknowledged	Triggered by confirmation action
Checked In	Candidate physically checked in at venue	Admin or kiosk check-in
Ongoing	Interview in progress	Triggered at slot start time
Completed	Interview concluded	Marked by employer or admin
Missed	Candidate did not appear	Auto-triggered if no check-in by cutoff
Cancelled	Interview removed	Admin or employer cancellation
Rescheduled	Moved to a new slot	Creates new Scheduled record

5.6 Event-Day Coordination Dashboard (Admin)

The admin dashboard must display in real time:

- Live interview status board — all interviews across all booths.
- Candidate check-in tracking — who has arrived, who is missing.
- Employer booth readiness indicators.
- Delay alerts — interviews running over time.
- Missed interview flags with one-click rescheduling.
- Active schedule timeline view (Gantt-style per booth).
- Global event progress metrics (completed / pending / missed counts).

5.7 Notifications & Reminder System

Supported channels: in-app notifications, email, optional SMS.

Notification Event	Delivery Trigger
Interview Confirmation	Sent to the candidate and the employer upon schedule generation
24-Hour Reminder	Sent to all confirmed participants the day before the event
2-Hour Reminder	Sent on event day, 2 hours before the interview slot
15-Minute Reminder	Sent on event day, 15 minutes before the slot
Slot Change Alert	Sent immediately upon any rescheduling or cancellation
Check-In Prompt	Sent to candidate upon arrival at venue (if GPS/QR enabled)
Missed Interview Alert	Sent to the admin when the candidate does not check in
Post-Event Summary	Sent to employers with outcome report after the event

6. Non-Functional Requirements

6.1 Performance

- Page load time under 2 seconds on standard broadband.
- Scheduling engine generates conflict-free schedules in under 3 seconds for up to 200 interviews.
- Real-time dashboard updates within 2-second latency.
- API response time under 300ms for all CRUD operations.

6.2 Scalability

- Architecture must support 500+ concurrent users during peak event hours.
- Horizontal scaling via containerised deployment (Docker + cloud).
- Database reads are optimised with indexing on schedule and candidate queries.
- The notification queue must handle 1,000+ messages per minute without blocking.

6.3 Security

- HTTPS is enforced across all endpoints.
- JWT tokens with short expiry (1 hour access, 7-day refresh).
- Role-based API guards on every endpoint — no endpoint is unauthenticated except login/register.
- Input sanitisation on all form fields to prevent XSS and SQL injection.
- File upload validation: type whitelist (PDF only for CVs), max size enforcement.
- Admin portal access restricted to whitelisted IP ranges (configurable).

6.4 Reliability & Offline

- 99.5% uptime target during event windows.
- Graceful degradation: if real-time socket drops, fall back to polling.
- Scheduled jobs use persistent queues (not in-memory) to survive restarts.
- All notification sends are idempotent — retry-safe.

6.5 Accessibility & Device Support

- Student dashboard: mobile-first, optimised for Android and iOS browsers.
- Employer dashboard: desktop-optimised, tablet-compatible.
- Admin dashboard: desktop, minimum 1280px viewport.
- Touch targets minimum 44px. WCAG AA colour contrast compliance.

7. Target Users and User Flows

7.1 Employer Flow

- Employer receives invite link → registers with company details.
- Submits job description, required skills, and available interview slots.
- Receives notification when an admin curates a candidate shortlist.
- Reviews shortlisted candidates and marks selected interviewees.
- Receives the generated interview schedule for their booth.
- On event day: views live schedule, marks interviews as completed.
- Post-event: receives outcome summary report.

7.2 Student / Developer Flow

- Registers via public link → completes profile and uploads CV.
- Awaits admin approval of registration.
- Receives notification of shortlisting by one or more employers.
- Receives confirmed interview schedule with booth locations and times.
- On event day: checks in at venue (QR or admin-assisted).
- Receives reminders and booth navigation info.
- Attends interviews; receives any last-minute updates via in-app alert.

7.3 Admin Flow

- Creates event (name, date, venue, booths, time window).
- Invites employers and reviews candidate registrations.
- Curates candidate pools per employer JD using filters.
- Monitors employer selections and triggers schedule generation.
- Reviews generated schedule; resolves any flagged conflicts manually.
- Publish schedule — notifications sent to all parties.
- On event day: monitors live dashboard, manages check-ins, handles exceptions.
- Post-event: runs reports, exports outcomes.

8. Technical Architecture

8.1 System Architecture Overview

The platform follows a three-tier architecture with a decoupled frontend and backend:

- Frontend: React (TypeScript) SPA — served via CDN.
- Backend: Django REST Framework — JSON API.
- Database: PostgreSQL — primary data store.
- Real-time Layer: Django Channels + WebSockets for live dashboard.
- Task Queue: Celery + Redis — async notifications and scheduled jobs.
- File Storage: S3-compatible cloud storage for CV uploads.
- Email: Transactional email via SendGrid or Postmark.
- SMS (optional): Africa's Talking or Twilio gateway.

8.2 Recommended Technology Stack

Layer	Technology	Notes / Libraries
Frontend	React 18 + TypeScript	Vite, React Query, Zustand, Tailwind CSS
Backend	Python 3.12 + Django 5	Django REST Framework, Django Channels
Database	PostgreSQL 16	With pgcrypto for sensitive fields
Cache / Queue	Redis 7	Celery for async tasks and notification queue
Real-time	WebSockets	Django Channels over ASGI (Daphne/Uvicorn)
Auth	JWT (djangorestframework-simplejwt)	Short-lived access + refresh tokens
Email	SendGrid / Postmark/Brevo	Transactional, template-based
Hosting	Railway / Render / AWS ECS	Containerised via Docker
CI/CD	GitHub Actions	Automated test + deploy pipeline
Monitoring	Sentry + UptimeRobot	Error tracking and uptime alerts

8.4 Scheduling Engine Logic

The scheduling engine runs as a backend service triggered after employer candidate selection:

- Load all selected (candidate, employer) pairs for the event.
- Load all available InterviewSlots per employer.
- Run constraint-satisfaction allocation:
 - Hard constraint: one candidate in one slot at one booth (no candidate overlap).
 - Hard constraint: one slot per employer booth at a time (no employer overlap).
 - Soft constraint: minimise candidate idle time between interviews.

- Soft constraint: respect candidate-declared unavailability windows.
- Generate Interview records with state = Scheduled.
- Flag unresolvable conflicts for admin review.
- Trigger confirmation notifications for all scheduled interviews.

Conflict prevention at write time: all slot assignments acquire an optimistic lock on the InterviewSlot row. Concurrent writes attempt a database-level unique constraint on (slot_id, candidate_id) and (slot_id, employer_id) pairs.

9. Frontend Modules

9.1 Admin Dashboard

- Event management: create/edit events, booths, and time windows.
- Candidate management: approve/reject, filter, curate shortlists.
- Schedule management: view/edit generated schedules, resolve conflicts.
- Live event board: real-time interview status grid (booth × time slot).
- Check-in console: mark candidate arrivals, trigger missed-interview flags.
- Notifications console: view sent/pending notifications, trigger manual sends.
- Reports: post-event summary tables, exportable as CSV.

9.2 Employer Dashboard

- JD submission form: title, skills, slot duration, capacity.
- Availability setter: drag-select or form-based slot definition.
- Candidate shortlist review: card view per candidate with CV link.
- Candidate selection: one-click select/deselect with running count.
- Schedule view: timeline view of confirmed interviews at own booth.
- Interview status: mark Completed or flag No-Show per interview.

9.3 Student / Developer Dashboard

- Profile form: name, email, phone, skills, experience level, portfolio links.
- CV upload: drag-and-drop or file picker, upload progress indicator.
- Schedule view: list of confirmed interviews, sorted by time, with booth info.
- Notification inbox: in-app alerts for all schedule-related events.
- Mobile-first layout: single-column, large tap targets, bottom navigation bar.

10. MVP Scope (10-Day Delivery Target)

The MVP must deliver operational reliability over feature completeness. The following scope is fixed for the 10-day sprint:

Feature
Authentication (all 3 roles)
Employer JD submission
Candidate registration + CV upload
Admin candidate approval and curation
Employer candidate selection
Automated schedule generation (constraint engine)
Conflict detection and admin conflict resolution UI
Interview state management (all 8 states)
Admin live dashboard (polling-based, not WebSocket)
Email notifications (confirmations, reminders)
In-app notifications
Employer schedule view
Student schedule view (mobile)
Manual rescheduling by admin